Simulación de Variables Aleatorias Discretas mediante el Método de la Transformada Inversa

Juan Felipe Rodríguez G. (20261595004)

3 de marzo de 2026

Introducción

La simulación de variables aleatorias es fundamental en el análisis estadístico y la validación de modelos matemáticos. En el contexto de las Ciencias de la Información, generar observaciones sintéticas que sigan distribuciones de probabilidad específicas permite evaluar el comportamiento de sistemas complejos sin necesidad de experimentación costosa o impráctica [1].

El presente trabajo implementa un generador de variables aleatorias discretas en MATLAB/GNU Octave, utilizando el método de la transformada inversa. Este método garantiza que las muestras generadas respeten exactamente la distribución de probabilidad especificada. La validación se realiza mediante simulación Monte Carlo, verificando la convergencia de las frecuencias relativas hacia las probabilidades teóricas.

Marco Teórico

Variable Aleatoria Discreta

Una variable aleatoria discreta X es una función que asigna un valor numérico a cada resultado de un espacio muestral discreto Ω [1]. Su comportamiento se caracteriza completamente por su función de masa de probabilidad (PMF):

$$p(x_i) = P(X = x_i), \quad i = 1, 2, \ldots, n$$

sujeta a las restricciones:

$$p(x_i) \geq 0 \quad \text{y} \quad \sum_{i=1}^{n} p(x_i) = 1$$

Método de la Transformada Inversa

El teorema de la transformada inversa establece que si U es una variable aleatoria con distribución uniforme en $(0, 1)$ y F es la función de distribución de una variable aleatoria X, entonces:

$$X = F^{-1}(U) = \text{mín}\{x : F(x) \geq U\}$$

tiene distribución F [1].

Para el caso discreto, el algoritmo consiste en:

1. Generar $u \sim \text{Uniforme}(0, 1)$
2. Calcular los intervalos acumulados: $I_i = \sum_{k=1}^{i} p(x_k)$
3. Retornar x_i donde $I_{i-1} < u \leq I_i$

Este método garantiza que $P(X = x_i) = p(x_i)$, preservando exactamente la distribución especificada.

Implementación

Función `va(x,p)`

La función recibe dos parámetros:

- **x**: Vector con los valores posibles de la variable aleatoria
- **p**: Vector con las probabilidades asociadas a cada valor

Y retorna un elemento de x seleccionado aleatoriamente según la distribución p.

Lógica de la Solución

La implementación sigue estos pasos fundamentales:

1. Determinación automática del tamaño: Se utiliza la función `length(x)` para identificar dinámicamente la cantidad de elementos, permitiendo que la función opere con vectores de cualquier dimensión.

2. Normalización de probabilidades: Se divide cada elemento de p por su suma total para garantizar $\sum p_i = 1$, evitando errores por aproximación numérica.

3. Generación de número aleatorio: Se genera $u \sim \text{Uniforme}(0, 1)$ mediante `rand()`.

4. Comparación con intervalos acumulados: Se utiliza una estructura `if-elseif-else` para comparar u con los intervalos de probabilidad acumulada, similar al método visto en clase para simular dados y eventos. Esta estructura determina a qué intervalo pertenece el número aleatorio generado.

Código de la Función

```
function valor = va(x, p)
    % Determinar automaticamente la cantidad de elementos
    n = length(x);

    % Normalizar probabilidades para que sumen 1
    p = p / sum(p);

    % Generar numero aleatorio uniforme entre 0 y 1
    u = rand;

    % Calcular intervalos acumulados y buscar donde cae u
    % Primer intervalo: [0, p(1))
    if u < p(1)
        valor = x(1);
    % Segundo intervalo: [p(1), p(1)+p(2))
    elseif u < (p(1) + p(2))
        valor = x(2);
    % Para el resto de elementos, calcular la suma acumulada
    else
        suma_acumulada = p(1) + p(2);
        for i = 3:n
            if u < (suma_acumulada + p(i))
                valor = x(i);
                return;
            end
            suma_acumulada = suma_acumulada + p(i);
        end
```

```
        % Si no cay  en ninguno anterior, es el  ltimo
        valor = x(n);
    end
end
```

Listing 1: Función va(x,p) - Generador de variables aleatorias discretas

La función emplea una lógica similar a la implementación del dado y los partidos vista en clase, donde se compara un número aleatorio con intervalos predefinidos para determinar el resultado.

Validación mediante Simulación

Para verificar el correcto funcionamiento, se desarrolló un script de validación que estima frecuencias relativas mediante $N = 50\,000$ ejecuciones independientes. Esta metodología se fundamenta en la Ley de los Grandes Números.

Ley de los Grandes Números

Teorema: Sea $X_1, X_2, \ldots, X_N$ una sucesión de variables aleatorias independientes e idénticamente distribuidas. Si definimos la frecuencia relativa como:

$$\hat{p}_j = \frac{\text{cantidad de veces que } X = x_j}{N}$$

entonces:

$$\lim_{N\to\infty} \hat{p}_j = p_j \quad \text{(casi seguramente)}$$

Esto significa que al realizar muchas simulaciones, las frecuencias observadas convergen a las probabilidades teóricas.

Ejemplos Implementados

Se validaron tres casos de estudio diferentes:

Ejemplo 1: Dado Justo

Distribución uniforme discreta con:

$$x = \{1, 2, 3, 4, 5, 6\}, \quad p_i = \frac{1}{6} \quad \forall i$$

Este caso representa un dado equilibrado donde todos los resultados son equiprobables, similar al ejercicio visto en clase.

Ejemplo 2: Moneda Sesgada

Distribución de Bernoulli con:

$$x = \{0, 1\}, \quad p = (0{,}7, 0{,}3)$$

donde 0 representa "cara" (70 %) y 1 representa "sello" (30 %). Este caso ilustra una distribución asimétrica.

Ejemplo 3: Distribución Arbitraria

Distribución no uniforme con:

$$x = \{10, 20, 30, 40, 50\}, \quad p = (0{,}1, 0{,}15, 0{,}35, 0{,}25, 0{,}15)$$

Este ejemplo demuestra la versatilidad del método para cualquier distribución discreta.

Estructura del Script de Validación

El script emplea la misma estructura vista en clase para el ejercicio `pruebaPartido()`:

1. Se inicializan contadores en cero
2. Se ejecuta un `for` con N repeticiones
3. En cada iteración se llama a `va(x,p)` y se incrementa el contador correspondiente usando `if-elseif`
4. Se calculan las frecuencias relativas: `frecuencias = conteo / N`
5. Se generan gráficas con barras comparando probabilidades teóricas vs. frecuencias observadas
6. Se personalizan colores y se agregan etiquetas con porcentajes

Fragmento del Script de Validación

```
% Inicializar contadores
conteo1 = [0, 0, 0, 0, 0, 0];

% Realizar simulaciones
for i = 1:N
    resultado = va(x1, p1);

    % Contar cual valor salio
    if resultado == 1
        conteo1(1) = conteo1(1) + 1;
    elseif resultado == 2
        conteo1(2) = conteo1(2) + 1;
    elseif resultado == 3
        conteo1(3) = conteo1(3) + 1;
    elseif resultado == 4
        conteo1(4) = conteo1(4) + 1;
    elseif resultado == 5
        conteo1(5) = conteo1(5) + 1;
    else
        conteo1(6) = conteo1(6) + 1;
    end
end

% Calcular frecuencias relativas
frecuencias1 = conteo1 / N;

% Graficar resultados
figure;
b1 = bar([p1; frecuencias1]');
xlabel('Valor del dado');
ylabel('Probabilidad');
title(['Dado justo - ' num2str(N) ' lanzamientos']);
legend('P(teorica)', 'Frecuencia observada');
grid on;
```

Listing 2: Extracto del script de validación - similar a lo visto en clase

Resultados

Al ejecutar el script de validación con $N = 50\,000$ simulaciones, se observa que:

1. **Convergencia numérica:** Los errores absolutos $|p_i - \hat{p}_i|$ son típicamente menores a 0,005, confirmando la convergencia.

2. **Consistencia visual:** Las gráficas de barras muestran una superposición prácticamente perfecta entre las probabilidades teóricas y las frecuencias observadas.

3. **Validación del método:** Los tres ejemplos confirman que la función `va(x,p)` respeta las distribuciones especificadas.

Gráficas de Validación

Las siguientes gráficas comparan las probabilidades teóricas (azul) con las frecuencias relativas observadas (naranja) para cada uno de los tres ejemplos implementados:

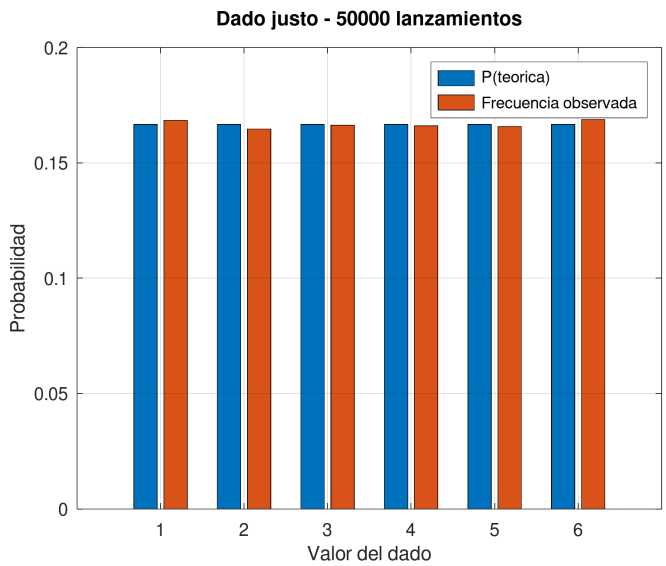

Figura 1: Ejemplo 1: Dado justo con distribución uniforme. Se observa que las seis caras tienen frecuencias relativas cercanas a $1/6 \approx 0{,}1667$.

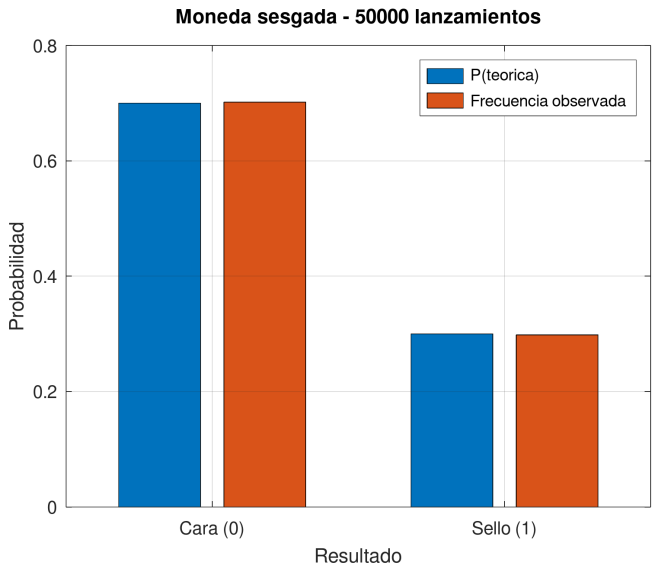

Figura 2: Ejemplo 2: Moneda sesgada con $P(\text{Cara}) = 0{,}7$ y $P(\text{Sello}) = 0{,}3$. La asimetría de la distribución se respeta correctamente.

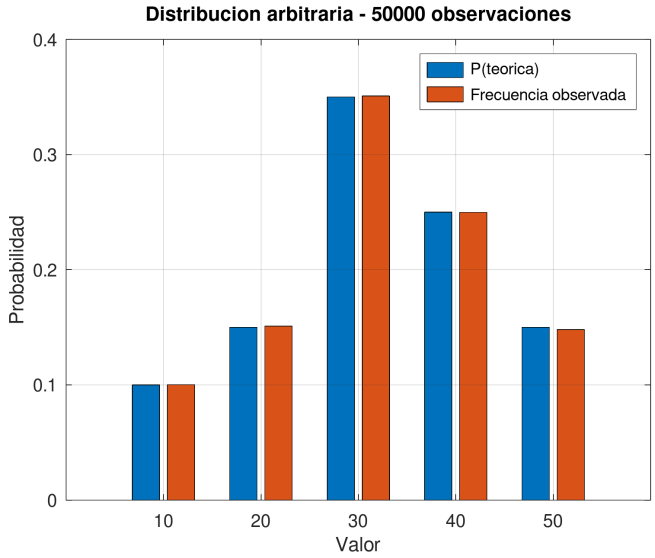

Figura 3: Ejemplo 3: Distribución arbitraria no uniforme con cinco valores. Las frecuencias observadas replican fielmente las probabilidades especificadas, demostrando la versatilidad del método.

En las tres figuras se aprecia la superposición prácticamente perfecta entre las barras azules (probabilidades teóricas) y las barras naranjas (frecuencias observadas), validando empíricamente la correcta implementación del método de la transformada inversa.

Conclusiones

La implementación desarrollada constituye una herramienta fundamental para la simulación de variables aleatorias discretas. El método de la transformada inversa, implementado mediante una estructura simple de

`if-elseif` similar a los ejercicios vistos en clase, proporciona una solución computacionalmente eficiente y conceptualmente clara.

La validación empírica mediante simulación Monte Carlo confirma que la función `va(x,p)` respeta rigurosamente las distribuciones de probabilidad especificadas, cumpliendo con el objetivo de aprendizaje del curso: validar modelos matemáticos mediante técnicas de simulación estadística.

Este ejercicio ilustra cómo conceptos teóricos de probabilidad pueden traducirse en algoritmos computacionales funcionales utilizando estructuras de programación básicas. La Ley de los Grandes Números se manifiesta empíricamente en la convergencia de las frecuencias observadas hacia las probabilidades teóricas.

Finalmente, la implementación es compatible tanto con MATLAB como con GNU Octave, garantizando accesibilidad y reproducibilidad en diferentes plataformas.

Instrucciones de Uso

El archivo `Tarea5_JuanFelipeRodriguezG.m` contiene tanto la función `va(x,p)` como el script de validación completo.

Para ejecutar la validación completa:

```
>> validar_tarea5
```

Esto ejecutará los tres ejemplos, mostrará las tablas de resultados y generará las gráficas correspondientes.

Para usar la función directamente:

```
>> lanzarDado = va([1,2,3,4,5,6], [1/6,1/6,1/6,1/6,1/6,1/6])
>> moneda = va([0,1], [0.7,0.3])
>> resultado = va([10,20,30,40,50], [0.1,0.15,0.35,0.25,0.15])
```

En GNU Octave desde terminal:

```
$ octave Tarea5_JuanFelipeRodriguezG.m
```

Referencias

[1] Walpole, R. E., Myers, R. H., Myers, S. L., & Ye, K. (2017). *Probability and Statistics for Engineers and Scientists* (9th ed.). Pearson.